Set_A_Lab5 - Results

Attempt 1 of 1

Written Feb 11, 2026 11:33 AM - Feb 11, 2026 12:18 PM

Attempt Score	**12.5 / 13.5 - 92.59 %**
Overall Grade (Highest Attempt)	**12.5 / 13.5 - 92.59 %**

Question 1

Use cocalc.com to run the code given below and answer the following questions(Q2, Q3, Q4, Q5, Q6):

```
gcc main1.c -lpthread -o main1.out
./main1.out & pstree -p | grep main1.out
```

main1.c

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
int fd, counter = 0;
void *myMethod(void *arg){
    int * id;
    id = (int *)arg;
    char st[]=" [%c] Hello from thread_%d.\n";
        for(int i=0; i<5; i++ ){
            sprintf(st, st, *id==1? '*': '+', *id);
            write(fd, st, strlen(st));
        }
        printf("Thread %d counter = %d \n", *id, ++counter);
        sleep(1);
    return NULL;
}

int main(int argc, char *argv[]){
    fd = open("myfile.txt",'a');
    if (fd<0){
        printf("If you see this message, delete myfile.txt using rm -f myfile.txt, then run the program again.\n");
        exit(0);
    }
    pthread_t threadMyMethodOne;
    pthread_t threadMyMethodTwo;
    int first_thread = 1, second_thread = 2;
    pthread_create(&threadMyMethodOne, NULL, myMethod, &first_thread);
    pthread_create(&threadMyMethodTwo, NULL, myMethod, &second_thread);
    pthread_join(threadMyMethodOne, NULL);
    pthread_join(threadMyMethodTwo, NULL);
    printf("The file (fd=%d) is  closed.\n", fd);
    close(fd);
    return 0;
}
```

Insert a screenshot of your output.

```
~$ mkdir -p week6 && cd week6
~/week6$ nano main1.c
~/week6$ rm -f myfile.txt
~/week6$ gcc main1.c -lpthread -o main1.out
~/week6$ ./main1.out & pstree -p | grep main1.out
[1] 584
Thread 2 counter = 1
Thread 1 counter = 2
                          |           |-main1.out(584)-+-{main1.out}(587)
                          |           |                `-{main1.out}(588)
~/week6$ The file (fd=3) is closed.
```

Screenshot 2026-02-11 at 11.45.53 AM.png (55.47 KB)

The correct answer is not displayed for Written Response type questions.

Question 2

After running the program, insert a screenshot showing the contents of myfile.txt. You may need to change the file's permissions using a command like **chmod 444 myfile.txt**.

```
~/week6$ ./main1.out
Thread 1 counter = 1
Thread 2 counter = 2
The file (fd=3) is closed.
~/week6$ cat myfile.txt
[*] Hello from thread_1.
[+] Hello from thread_2.
[*] Hello from thread_1.
[+] Hello from thread_2.
[*] Hello from thread_1.
[*] Hello from thread_1.
[+] Hello from thread_2.
[*] Hello from thread_1.
[+] Hello from thread_2.
[+] Hello from thread_2.
~/week6$
```

Screenshot 2026-02-11 at 11.49.46 AM.png (46.36 KB)

The correct answer is not displayed for Written Response type questions.

Question 3

Based on the output of your code, do you think that the global variable "counter" is shared between threads 1 and 2 (Y/N)?

___Y___ ✔(50 %)

From the contents of myfile.txt, is the file myfile.txt shared between the two threads (Y/N)? ___Y___ ✔(50 %)

Question 4

Change the sleep time from 1 to 100, then recompile your program and run it using

./main1.out & pstree -p | grep main1.out

From the output, Obtain the *pid* and *tid* for the process and threads. Insert a screenshot of your output.

```
~/week6$ ./main1.out & pstree -p | grep main1.out
[1] 809
Thread 2 counter = 1
Thread 1 counter = 2
                          |           |-main1.out(809)-+-{main1.out}(812)
                          |           |                `-{main1.out}(813)
~/week6$
```

Screenshot 2026-02-11 at 11.54.38 AM.png (42.7 KB)

Screenshot 2026-02-11 at 11.56.53 AM.png (32.28 KB)

The correct answer is not displayed for Written Response type questions.

Question 5

After running (make sure the sleep time is already changed from 1 to 100)

./main1.out & pstree -p | grep main1.out

In the second terminal use **kill -9 <tid>** to terminate one of the threads. Does the main process remain alive after the thread is killed (Y/N)? ___Y___ ✖ **(N)**

Question 6

If a process creates two threads, which of the following statements is true?

- ○ Each thread receives a completely new PID and no TID
- ○ Both threads have neither PID nor TID
- ○ Both threads have different PIDs but identical TIDs
- ✔ ◉ Both threads share the same PID but have different TIDs

Question 7

Use the online compiler https://www.onlinegdb.com/online_c_compile to run user_level.c and then answer the following questions (Q8 and Q9).

user_level.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <ucontext.h>
#include <sys/syscall.h>

#define STACK_SIZE 8192
ucontext_t main_ctx, th1_ctx, th2_ctx;


void thread_1() {
   int c = 0;
   while(1){
      c++;
      printf("Thread 1.\n");
      if (c%5==0){
         sleep(1);
         swapcontext(&th1_ctx, &th2_ctx);
      }
   }
}

void thread_2() {
   int c = 0;
   while(1){
      c++;
      printf("Thread 2.\n");
      if (c%3==0){
         sleep(1);
         swapcontext(&th2_ctx, &th1_ctx);
      }
   }
}
```

```
int main() {
    // Allocate stacks
    char *sum_stack = malloc(STACK_SIZE);
    char *sort_stack = malloc(STACK_SIZE);

    if (!sum_stack || !sort_stack) {
        perror("malloc");
        exit(1);
    }

    // Create sum thread context
    getcontext(&th1_ctx);
    th1_ctx.uc_link = 0;
    th1_ctx.uc_stack.ss_sp = sum_stack;
    th1_ctx.uc_stack.ss_size = STACK_SIZE;
    makecontext(&th1_ctx, thread_1, 0);

    // Create sort thread context
    getcontext(&th2_ctx);
    th2_ctx.uc_link = 0;
    th2_ctx.uc_stack.ss_sp = sort_stack;
    th2_ctx.uc_stack.ss_size = STACK_SIZE;
    makecontext(&th2_ctx, thread_2, 0);

    setcontext(&th1_ctx);

    free(sum_stack);
    free(sort_stack);
    return 0;
}
```

Insert a screenshot of your output.

main.c

```c
        exit(1);
    }

    // Create sum thread context
    getcontext(&th1_ctx);
    th1_ctx.uc_link = 0;
    th1_ctx.uc_stack.ss_sp = sum_stack;
    th1_ctx.uc_stack.ss_size = STACK_SIZE;
    makecontext(&th1_ctx, thread_1, 0);

    // Create sort thread context
    getcontext(&th2_ctx);
    th2_ctx.uc_link = 0;
    th2_ctx.uc_stack.ss_sp = sort_stack;
    th2_ctx.uc_stack.ss_size = STACK_SIZE;
    makecontext(&th2_ctx, thread_2, 0);

    setcontext(&th1_ctx);

    free(sum_stack);
    free(sort_stack);
    return 0;
}
```

input

```
Thread 1.
Thread 1.
Thread 2.
Thread 2.
Thread 2.
Thread 1.
Thread 1.
Thread 1.
Thread 1.
Thread 1.
Thread 2.
Thread 2.
Thread 2.
Thread 1.
Thread 1.
Thread 1.
Thread 1.
Thread 1.
```

Screenshot 2026-02-11 at 12.06.40 PM.png (285.28 KB)

The correct answer is not displayed for Written Response type questions.

Question 8

Which thread starts first? e.g. 1 or 2 ___1___ ✔(25 %)

Modify thread_1() and thread_2() to print the thread ID using

```
printf("Thread 1.\n");  ⇝  printf("Thread 1. PID: %d TID: %ld \n", getpid(), syscall(SYS_gettid));
printf("Thread 2.\n");  ⇝  printf("Thread 2. PID: %d TID: %ld \n", getpid(), syscall(SYS_gettid));
```

Does each thread have a different thread ID (Y/N)? ___N___ ✔(25 %) Change the sleep(1) calls to sleep(100) in both thread functions.

Does one sleep call block both threads (Y/N)? ___Y___ ✔(25 %)

Does one sleep call block the main process (Y/N)? ___Y___ ✔(25 %)

Question 9

A process uses two user-level threads managed entirely in user space. Which of the following statements is true regarding the PID and TIDs seen by the Linux kernel?

- ✔ (●) The process has one PID, and both user-level threads share the same TID as the PID
- ( ) Each user-level thread has a unique TID, but they all share the same PID
- ( ) Each user-level thread has no TID, only the PID
- ( ) The kernel assigns a different PID to each user-level thread

Question 10

Use the online C compiler

https://www.onlinegdb.com/online_c_compiler

to run your code and then answer the following questions.

```c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <ctype.h>
#include <sys/wait.h>

int global_var = 0;

int main(){
  int status;
  int pid_t, fork_return;
  fork_return = fork();

  if (fork_return==0){
    global_var++;
    printf("Child  pid  = %d \t global_var = %d\n", getpid(), glob
  }else{
    global_var++;
    waitpid(-1, &status, 0);
    printf("Parent pid  = %d \t global_var = %d\n", getpid(), glob
  }
  return 0;
}
```

Insert a screenshot of your output.

```c

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <ctype.h>
#include <sys/wait.h>

int global_var = 0;

int main() {
    int status;
    int pid_t, fork_return;

    fork_return = fork();

    if (fork_return == 0) {
        global_var++;
        printf("Child pid = %d \t global_var = %d\n", getpid(), global_var);
    } else {
        global_var++;
        waitpid(-1, &status, 0);
        printf("Parent pid = %d \t global_var = %d\n", getpid(), global_var);
    }

    return 0;
}
```

input

```
Child pid = 3614         global_var = 1
Parent pid = 3610        global_var = 1


...Program finished with exit code 0
Press ENTER to exit console.
```

Screenshot 2026-02-11 at 12.15.56 PM.png (264.12 KB)

The correct answer is not displayed for Written Response type questions.

Question 11

From the output, determine the value of the global_var variable in the parent process. e.g. 2

___1___ ✔(33.33 %)

From the output, determine the value of the global_var variable in the child process. e.g. 2

___1___ ✔(33.33 %)

Based on your observations, do you think that the global_var variable is shared between the parent and child processes (Y/N)? ___N___ ✔(33.33 %)

Done